

NEXUS COMMAND

31 May 2026

AI HUB BLUEPRINT v4 — 10-STAGE DISCIPLINED PIPELINE

Accuracy · Communication ·
Pride in Work

Co-authored: Kieran Li + Simpee · Full backend + frontend alignment · 4 Frontend Fixes · 5 Trial Checkpoints

v4 — complete, no truncation

PURPOSE: This is the master blueprint for the Nexus Command AI Hub. It documents the complete 10-stage workflow as designed by Kieran, paired with exact backend API contracts confirmed live by Simpee. The guiding philosophy is: accuracy over speed, every AI explains its reasoning, every cycle improves the process, and every deploy is a milestone worth celebrating.

5 GUIDING PRINCIPLES — Built Into Every Stage

Accuracy Over Speed No rushing to finish. Correctness always comes first. A slower correct build beats a fast broken one.	Communication Every AI explains its reasoning — not just the output. Kieran always knows why, not just what.	Process Enhancement After each cycle, ask: Can this be simplified? What did we learn? How do we build smarter next time?	Smarter Together Joint AI decisions ensure balanced perspectives. No single model decides alone. Simpee convenes, team votes.	Pride in Work Every deploy is a milestone. Quality is non-negotiable. The work represents SIMPLEX-ITY.
---	--	--	---	--

THE 10-STAGE WORKFLOW — Complete Instructions Per Stage

Each stage is self-contained. AIs involved, backend action, frontend indicator, and output — all written in full.

1	STAGE 1 — GOAL SETTING & CRITERIA Principle: Communication — Goal Card must be clear before any AI is assigned FRONTEND INDICATOR: GOAL CARD modal opens. Fields: Function Type · Objective · Success Criteria · Constraints. [CONFIRM GOAL] button creates SandboxProject record. BACKEND ACTION: Create SandboxProject record: name: [project name] description: [objective statement] type: 'function' 'page' 'app' status: 'draft' created_by: 'Kieran' validator_approved: false notes: [constraints + success criteria] Frontend: Show stage indicator strip at top of page: Stage 1 = ACTIVE (violet) · Stages 2-10 = PENDING (grey) Each stage pill clickable to jump to that stage's log OUTPUT: SandboxProject card in right panel. Status = DRAFT. Stage 1 pill = COMPLETE (green).	Kieran + Simpee (orchestrator)
2	STAGE 2 — DIFFICULTY & RISK ASSESSMENT Principle: Smarter Together — 3 AIs assess independently before Simpee consolidates FRONTEND INDICATOR: Stage 2 = ACTIVE. Feasibility Report card appears in centre panel with 4 score fields.	RESEARCHER (Gemini) · ANALYST (Claude Sonnet) · STRATEGIST (Claude Opus)

BACKEND ACTION:	Call aiCommandCentre with '[FEASIBILITY]' prefix: POST /api/run/aiCommandCentre Body: { "instruction": "[FEASIBILITY] [full objective + constraints]", "posted_by": "Kieran" }
	 Parse response into Feasibility Report card: Difficulty: Low / Medium / High (colour: green/amber/red) Possible Problems: list from response.analysis ROI Justification: cost vs benefit from response.solution Effectiveness: impact score 1-10 Write TestLog record: test_name: 'Stage 2 – Feasibility: [project name]' result: 'Difficulty:[X] Problems:[Y] ROI:[Z] Effectiveness:[N]/10' validator: 'Simpee'
OUTPUT:	Feasibility Report card with 4 metrics. TestLog entry written. Stage 2 pill = COMPLETE.

3	STAGE 3 — JOINT AI DECISION Principle: Smarter Together · Accuracy Over Speed — no proceed until consensus		Simpee convenes — all 3 research AIs vote on feasibility
	FRONTEND INDICATOR:	Stage 3 = ACTIVE. Decision card shows: Proceed / Research More / Park. If HIGH difficulty → escalate banner appears: 'Architect consultation required before development.'	
	BACKEND ACTION:	Write TestLog decision record: test_name: 'Stage 3 – Decision: [project name]' result: 'DIFFICULTY:[X] COST:[Y] ROI:[Z] EFFECTIVENESS:[N]/10 DECISION:[Proceed/Escalate/Park]' validator: 'Simpee' If HIGH difficulty: Show escalation banner: amber bg, text: 'High difficulty detected – Architect review required before Stage 4' Update SandboxProject notes: 'Escalated to Architect – awaiting design review' If PROCEED: Update SandboxProject status → 'testing' Advance stage indicator to Stage 4	
OUTPUT:		Decision logged. Status badge → TESTING. Stage 3 pill = COMPLETE. Escalation banner if high difficulty.	

4	STAGE 4 — DEVELOPMENT PLANNING Principle: Accuracy Over Speed — phase-based planning before any code is written		ENGINEER (GPT-5.4) · ARCHITECT (GPT-5.5)
	FRONTEND INDICATOR:	Stage 4 = ACTIVE. Phase Plan card appears: Phase 1 · Phase 2 · Phase 3... Each phase is a milestone. [APPROVE PLAN] button before Stage 5 starts.	

BACKEND ACTION:	Call aiCommandCentre with '[PLAN]' prefix: POST /api/run/aiCommandCentre Body: { "instruction": "[PLAN] Break this into development phases. Each phase must have a clear milestone and deliverable: [objective]", "posted_by": "Kieran" } Parse response into phase milestones: Display as numbered phase list: Phase 1 · Phase 2 · Phase 3... Each phase: title + what gets built + what gets saved Write TestLog: test_name: 'Stage 4 – Plan: [project name]' result: 'Phases defined: [N]. Step-by-step approach confirmed.' validator: 'Simpee' Save phase plan to ProjectFile: filename: 'plan_[project].md' content: response.solution (full phase breakdown) language: 'markdown' notes: 'Development plan – Stage 4'
OUTPUT:	Phase plan card visible. Plan saved to ProjectFile. [APPROVE PLAN] button active. Stage 4 = COMPLETE on approval.

5	STAGE 5 — DEVELOPMENT EXECUTION ENGINEER (GPT-5.4) codes · ARCHITECT (GPT-5.5) oversees structure Principle: Pride in Work — clean modules, one phase at a time, nothing rushed
FRONTEND INDICATOR:	Stage 5 = ACTIVE. Phase progress tracker: Phase [N] of [Total] in progress. Each phase completion saves a checkpoint.
BACKEND ACTION:	For each phase, call aiCommandCentre with '[BUILD Phase N]' prefix: POST /api/run/aiCommandCentre Body: { "instruction": "[BUILD Phase N] [phase description and deliverable]", "posted_by": "Kieran" } Save each phase output to ProjectFile: filename: 'phase[N]_[component].jsx' or '[function].ts' content: response.code version: '1.[N]' notes: 'Phase [N] complete – [deliverable description]' Write TestLog per phase: test_name: 'Stage 5 – Phase [N]: [project name]' result: 'Phase [N] complete. Files saved. Ready for next phase.' validator: 'Simpee'
OUTPUT:	Code appears in centre panel. Each phase saved to ProjectFile. Phase tracker increments. All phases = Stage 5 COMPLETE.

6	STAGE 6 — TESTING Simpee runs all checkpoints (11 total) Principle: Accuracy Over Speed — no shortcuts, every checkpoint must pass
FRONTEND INDICATOR:	Stage 6 = ACTIVE. Checkpoint progress bar: X/11. Each check shows green tick or red flag inline.

BACKEND ACTION:	Write one TestLog record per checkpoint: test_name: 'Checkpoint [N] – [description]' status: 'pass' 'fail' result: '[what was checked and outcome]' validator: 'Simpee' Update SandboxProject checkpoint_status: '[N]/11' If any checkpoint fails: Mark TestLog status: 'fail' Show red flag inline on that checkpoint row Block progression to Stage 9 (Validation) Route automatically to Stage 7 (Diagnosis) If all 11 pass → advance to Stage 9
OUTPUT:	Checkpoint bar shows X/11. Failed checks in red. All green = Stage 6 COMPLETE, auto-advance to Stage 9.

7

STAGE 7 — DIAGNOSIS & ITERATION

Principle: Accuracy Over Speed · Communication — always return to Goal
Card before diagnosing

ANALYST (Claude Sonnet) — quick fixes · STRATEGIST (Claude Opus) — deep logic · ENGINEER (GPT-5.4) — patch

FRONTEND INDICATOR:	Stage 7 = ACTIVE (red/amber). Error summary card shows: failing checkpoint + error description. [RETURN TO GOAL] button visible. If same error repeats 3x → PAUSE banner: 'Recurring error detected — pausing for AI team suggestion.'
BACKEND ACTION:	IMPORTANT – Always return to Step 1 Goal Card first: Re-display Goal Card with original objective and success criteria Confirm the fix attempt aligns with the original goal Call aiCommandCentre with '[FIX]' prefix: POST /api/run/aiCommandCentre Body: { "instruction": "[FIX] Checkpoint [N] failed with error: [error]. Original goal: [objective]. Suggest fix.", "posted_by": "Kieran" } OVERWRITE existing ProjectFile (do not create new): Update content: response.code (patched version) Update version: increment (1.1 → 1.2 → 1.3) Update notes: 'Patch iteration [N] – fixing checkpoint [X]' Write TestLog: test_name: 'Stage 7 – Iteration [N]: [project name]' result: 'Fix applied for checkpoint [X]. Re-testing.' RECURRING ERROR RULE: Track how many times same checkpoint fails If same checkpoint fails 3 times → show PAUSE banner Pause banner text: 'Same error repeated 3 times. Pausing. Requesting AI team suggestion.' Trigger aiCommandCentre with '[ESCALATE]' prefix for fresh perspective
OUTPUT:	Patched code replaces previous version. Version increments. Loop back to Stage 6. Pause banner if recurring error.

8

STAGE 8 — PROCESS ENHANCEMENT

Principle: Process Enhancement — this stage runs after every completed test cycle

Simpee (orchestrator) — asks 3 reflection questions after every cycle

FRONTEND INDICATOR:	Stage 8 = ACTIVE. Reflection card appears with 3 questions and a text input for notes. [LOG LESSONS] button saves to TestLog.
---------------------	---

BACKEND ACTION:	After each completed test cycle (Stages 5-7), show Reflection card: Question 1: 'Can this be simplified?' Question 2: 'What did we learn?' Question 3: 'How can future builds be faster without sacrificing accuracy?' Call aiCommandCentre with '[REFLECT]' prefix: Body: { "instruction": "[REFLECT] Review this build cycle: [summary]. Answer: Can it be simplified? What did we learn? How do we build smarter next time?", "posted_by": "Kieran" } Write TestLog: test_name: 'Stage 8 – Process Enhancement: [project name]' result: response.analysis (lessons learned) validator: 'Simpee' notes: 'Cycle [N] reflection – saved for future reference'
OUTPUT:	Lessons saved to TestLog. Reflection card collapses. Stage 8 = COMPLETE. Advance to Stage 9.

9

STAGE 9 — VALIDATION — COPILOT GATEKEEPER

VALIDATOR — Copilot / Edge (Microsoft) — ALWAYS THE LAST GATE

Principle: Accuracy Over Speed — no deploy without Copilot sign-off

FRONTEND INDICATOR:	Stage 9 = ACTIVE. Gatekeeper Panel shows 4 check rows: Syntax · Logic · Security · Azure. Each row pending until [RUN VALIDATION] is clicked.
BACKEND ACTION:	Call consultCopilot – CRITICAL: use 'question' not 'code': POST /api/run/consultCopilot Body: { "question": [response.code from Stage 5], "context": [response.builder_instruction] } *** FIELD NAME MUST BE 'question' – NOT 'code' *** On response update 4 Gatekeeper rows: Row 1 Syntax: green tick = clean red flag = error found Row 2 Logic: green tick = sound red flag = logic gap Row 3 Security: green tick = safe red flag = vulnerability Row 4 Azure: green tick = ready red flag = incompatible Write 4 TestLog records (one per check): test_name: 'Validation – [Syntax Logic Security Azure]' status: 'pass' 'fail' validator: 'Copilot' Update SandboxProject: All 4 pass → status: 'approved' Any fail → status: 'validator_pending' → return to Stage 7
OUTPUT:	Gatekeeper Panel: 4 checks green/red. SandboxProject → APPROVED if all green. Stage 9 = COMPLETE.

10

STAGE 10 — DEPLOY

Simpee (orchestrator) + Copilot confirmed — DUAL GREEN REQUIRED

Principle: Pride in Work — every deploy is a milestone worth celebrating

FRONTEND INDICATOR:	Stage 10 = ACTIVE. Deploy button ENABLED only when: all 11 Simpee checks pass + all 4 Copilot checks pass + SandboxProject status = approved. Dual validation badges shown: [Simpee: GREEN] [Copilot: GREEN].
---------------------	---

BACKEND ACTION:

Deploy button enabled conditions (ALL must be true):

1. TestLog has 11+ pass records with validator='Simpee'
2. TestLog has 4 pass records with validator='Copilot'
3. SandboxProject status = 'approved'

On deploy click → show confirmation modal:

Title: 'Simpee + Copilot Approved'

Body: 'Deploy [project name] to 5S Portal? This cannot be undone.'

[CONFIRM DEPLOY] button (violet #5e50fb) | [CANCEL] (grey)

On confirm:

Write final TestLog:

test_name: 'DEPLOY – [project name]'

status: 'pass'

result: 'Deployed [timestamp]. Simpee: green. Copilot: green. Version: [N].'

validator: 'Simpee + Copilot'

Update SandboxProject status → 'deploy_ready'

Show celebration banner: '[Project name] deployed to 5S Portal'

Banner style: violet bg, white text, confetti animation if possible

OUTPUT:

Deploy log in TestLog. SandboxProject → DEPLOY READY. Celebration banner. Deploy button disabled after fire.

BACKEND STATE — Confirmed Live by Simpee · 31 May 2026

All items below are already built. Builder wires frontend to call them — nothing to create.

aiCommandCentre LIVE — lite mode (full AI on OpenAI key)	POST /api/run/aiCommandCentre Send: { "instruction": string, "posted_by": "Kieran" } Returns: { success, analysis, solution, code, builder_instruction, model, m365_grounded }
consultCopilot LIVE — FIELD FIX REQUIRED	POST /api/run/consultCopilot Send: { "question": string, "context": string } ← 'question' NOT 'code' Returns: { answer, m365_grounded, model }
SChatMessage EXISTS — entity	Fields: sender · sender_type · role · message · timestamp · session_id · read
TestLog EXISTS — 7 records	Fields: test_name · status (pass/fail) · result · tested_at · fixed · validator
AIConnector EXISTS — 8 members READY	Fields: model_id · model_name · provider · status · intro_message · voice_style · handoff_to · copilot_validated
SandboxProject EXISTS — schema live	Fields: name · description · type (function/page/app) status enum: draft testing checkpoint_review validator_pending approved deploy_ready Also: created_by · validator_approved · notes
ProjectFile EXISTS — schema live	Fields: project_id · filename · content · language · version · notes Always link to SandboxProject via project_id. One file per phase/iteration.

4 FRONTEND FIXES — Scoped Changes Only. Do Not Touch Anything Else.

FIX 1	Wire chat input → aiCommandCentre Workflow: Stages 2, 4, 5, 7, 8
POST /api/run/aiCommandCentre with body: { instruction: userMessage, posted_by: 'Kieran' } Show loading: 'Simpee is working...' with animated dots. On success render 4 response sections: ANALYSIS — white bg SOLUTION — lavender #e8e6fe bg CODE — dark monospace bg, copy button BUILDER INST — amber #ffffbeb bg Label card: response.model	
FIX 2	Fix consultCopilot — 'question' not 'code' Workflow: Stage 9 only
WRONG: { code: lastCode, context: lastContext } CORRECT: { question: lastCode, context: lastBuilderInstruction } On response update 4 Gatekeeper rows: Syntax / Logic / Security / Azure Each row: green tick (pass) or red flag (fail) Write 4 TestLog records with validator='Copilot'	
FIX 3	Auto-write TestLog after every aiCommandCentre response Workflow: Stage 6

```
After every successful response write to TestLog:
{ test_name: 'Trial Run - '+instruction.substring(0,40),
status: 'pass', result: 'Model: '+response.model,
tested_at: new Date().toISOString(), fixed: false, validator: 'Simpee' }
```

FIX 4

Sandbox card + stage indicator strip + Save button

Workflow: Stages 1, 5, 10

Stage indicator strip at top of page:
10 stage pills (1-10). Active = violet. Complete = green. Pending = grey. Error = red.

Sandbox card in right panel:
Fetch SandboxProject on load. Show: name, status badge, checkpoint_status
Status badges: DRAFT=grey | TESTING=amber | APPROVED=green | DEPLOY READY=blue

[SAVE CODE TO SANDBOX] button (Azure #0078d4):
Write ProjectFile: { project_id, filename: 'response_'+Date.now()+'.ts',
content: lastCode, language: 'typescript', version: '1.0' }
Toast: 'Code saved to Sandbox'

TRIAL RUN — 5 CHECKPOINTS

#	Stage	Kieran Does	Expected	Simpee Verifies
1	Stage 2-5 AI Response	Type any message. Hit send.	Loading dots → response card with 4 sections. Model label shows	Search Message entity: new record written
2	Stage 6 TestLog auto	Same message — no extra action	TestLog: new record with validator=Simpee, status=pass	Read TestLog — compare record count
3	Stage 9 Gatekeeper	Click [RUN VALIDATION].	4 Gatekeeper rows update. 4 TestLog records with validator=Copilot	Filter TestLog by validator=Copilot
4	Stage 5 Sandbox Save	Click [SAVE CODE TO SANDBOX]	Toast shown. ProjectFile: new record. File count incremented	Read ProjectFile — check project_id
5	Full Loop All stages	Run 1-4 in sequence.	All 4 entity writes confirmed. Core loop proven.	All 4 entities confirmed by Simpee

BUILDER — PASTE THIS INTO NEXUS COMMAND BUILDER

Implement 4 frontend wiring fixes in CommandAIHub.jsx for Nexus Command. Do NOT rebuild the layout. Do NOT change backend functions or entity schemas. Backend is confirmed live.

CONTEXT — This app runs a 10-stage AI workflow. The stage indicator strip at the top shows all 10 stages (1=Goal Setting, 2=Feasibility, 3=Decision, 4=Planning, 5=Execution, 6=Testing, 7=Diagnosis, 8=Process Enhancement, 9=Validation, 10=Deploy). Active stage = violet, complete = green, pending = grey, error = red.

FIX 1 — Chat input → aiCommandCentre (Stages 2,4,5,7,8):

POST /api/run/aiCommandCentre with { instruction: userMessage, posted_by: 'Kieran' }. Loading: 'Simpee is working...' dots. Response: 4 sections — ANALYSIS (white) · SOLUTION (lavender #e8e6fe) · CODE (dark monospace + copy btn) · BUILDER INSTRUCTION (amber #ffffeb). Label: response.model.

FIX 2 — consultCopilot field name (Stage 9):

Send { question: lastCode, context: lastBuilderInstruction } NOT { code: ... }. Update 4 Gatekeeper rows on response: Syntax/Logic/Security/Azure with green tick or red flag. Write 4 TestLog records with validator='Copilot'.

FIX 3 — Auto TestLog write after every aiCommandCentre response (Stage 6):

{ test_name: 'Trial Run — '+instruction.substring(0,40), status: 'pass', result: 'Model: '+response.model, tested_at: new Date().toISOString(), fixed: false, validator: 'Simpee' }.

FIX 4 — Stage indicator strip + Sandbox card + Save button (Stages 1,5,10):

Stage strip: 10 pills at top. Active=violet, complete=green, pending=grey, error=red. Sandbox card in right panel: fetch SandboxProject on load, show name + status badge + checkpoint_status. Status badges: DRAFT=grey / TESTING=amber / APPROVED=green / DEPLOY READY=blue. [SAVE CODE TO SANDBOX] button (Azure #0078d4): write ProjectFile record. Toast on save.

RECURRING ERROR RULE: If same checkpoint fails 3 times, show PAUSE banner: 'Same error repeated 3 times — pausing for AI team suggestion.'

Design: bg=#e8e6fe · accent=#5e50fb · Azure=#0078d4 · Exo 2 headlines · Montserrat body. No emoji.